api0

GUIDE 01

How to Put Your Existing API in Claude, Teams and WhatsApp in Under 15 Minutes

Your application already does the work. Here's how to make it usable from Claude, Microsoft Teams and WhatsApp — no MCP server or bot to write — with every action taken as the person who asked.

Updated 7 November 2026 · Read online: api0.ai/guides/your-api-in-claude-teams-whatsapp

Your software has an API. It can create a ticket, pull an invoice, list today's jobs. But your users spend their day in Teams, on WhatsApp or in Claude. Between the two, there's no bridge.

Building that bridge yourself means writing an MCP server, handling OAuth for every user, storing your application's credentials somewhere the AI can't read them, logging every call — then doing it all again for Teams and for WhatsApp. Weeks of work.

This guide shows the short version with api0: import the API once, choose who can sign in, and connect the channels.

What you need

- An HTTP API reachable from the internet (REST/JSON).
- Its description: an OpenAPI spec (YAML or JSON), or failing that, the list of your endpoints.
- A Google account to sign in to app.api0.ai.
- For Teams: access to the Azure portal. For WhatsApp: a Meta Business account.

No API at hand to try it? Take one of the sample specs (CRM, ticketing, invoicing, field service, HR) and point it at a mock server.

Step 1 — Import the API (3 minutes)

1. Sign in at **app.api0.ai**.
2. **Tools** tab → import your OpenAPI spec.
3. Each operation becomes a **tool**: its name, description and parameters are what Claude sees.
4. A built-in test calls each tool before it counts as configured.

No spec? Describe your endpoints in plain English and api0 turns them into tools.

The tip that changes everything: write good descriptions. Claude picks a tool from its description. Say *when* to use it ("use before creating a contact, to avoid duplicates") and *what to confirm* before an

irreversible action ("ask for confirmation before sending an invoice"). A good description is worth ten lines of prompt.

A few details handled for you:

- parameter names MCP clients reject (`$filter`, `page[size]`) are exposed under a clean name (`filter`, `page_size`) and sent to your API under their real name;
- an error from your application comes back to Claude as a readable result, so it can correct and retry;
- a failed call is never billed.

Step 2 — Set up access (3 minutes)

Access tab. Two questions.

Who can sign in? By default, people sign in with Google. A company can plug in its own identity provider (Entra ID or any OIDC provider): its employees then sign in with their usual account.

Which credential is your application called with? This is the key question. The options, simplest to finest:

Option	For
Identity token signed by api0	Your own application: it verifies the call comes from api0 and for which workspace. No secret to store.
Fixed token or headers	A third-party API with a single key.
OAuth client credentials	An API that issues service tokens.
Each person's own credential	Each user adds their own token, or connects their account via OAuth: every action is taken **as them**, with **their** permissions.

In every case, the credential stays in api0. Claude never sees it.

A third setting: **who pays for paid tools** — the workspace, or each person on their own plan.

Step 3 — Claude (2 minutes)

Channels tab → **MCP**. You'll find your workspace's MCP server URL and a Client ID.

In claude.ai → Settings → Connectors → **Add custom connector**:

1. paste the server URL;
2. choose **"Use your own OAuth client"** and paste the **Client ID** (not the URL), with no secret;
3. sign in.

That's it. Ask Claude "what tools do you have?", then ask a real question. For a team, create a **Claude Project** with domain instructions (a template is provided).

Cursor, Claude Desktop and other MCP clients connect the same way.

Step 4 — Teams (5 minutes, on the Azure side)

Microsoft doesn't let a third party set up a bot for you: you need an **Azure Bot** resource in your name. api0 does the rest.

1. Azure portal → create an **Azure Bot** → note the App ID, create a secret.
2. app.api0.ai → Channels → **Teams**: paste the App ID, secret and tenant ID. api0 checks with Microsoft that they work and gives you the **messaging endpoint**.
3. Paste that endpoint into the Azure bot's configuration and enable the Teams channel.
4. Download the **Teams app package** api0 generates (manifest and icons) and install it.

The full walkthrough is in Put Your Product in Teams Without Building a Bot.

Step 5 — WhatsApp or Telegram (2 minutes)

Telegram: create a bot with @BotFather and paste the token into Channels → Telegram. api0 checks the token and sets up the webhook itself.

WhatsApp: in Channels → WhatsApp, enter the Phone Number ID, a permanent token and a verify token of your choice. Then paste the Webhook URL api0 gives you into your Meta app's WhatsApp configuration and subscribe to messages.

Either way, you can give the bot a system prompt (short answers, confirm before acting…). On their first message, each person links their account once; after that they talk to your API like a colleague.

What you get

- **One setup** for Claude, Teams, WhatsApp and Telegram: the same tools everywhere.
- **Every action taken as the person**, with their permissions in your system.
- **No secrets in the AI**: credentials stay encrypted in api0.
- **A log of every call** (Activity tab) and credits per call.
- **Nothing to host**: no MCP server, no bot, no new interface.

FAQ

My API is private, behind a VPN. api0 needs to reach it. Put it behind a gateway that only accepts calls signed by api0 (the "signed identity token" option).

I want to limit what the AI can do. Import only the operations that are useful, and prefer each person's own credential: your application then applies its own permissions.